```kotlin
fun main() {  new *
    for (i in 1 ≤ .. ≤ 10) {
        println("Hello, welcome to the tutorial")
    }
    TODO( reason: "Write some cool code so that they are impressed")
}
```

Programming Tutorial - 27

8. Tutorial - Generics & Recursions

Kuzey Aydınhan | 16. January 2026

Contents

1. Overview

2. Generics

3. Recursion

4. Revision

Information

- Homework 5 deadline: 29.01.

Important

The Präsenzübung is on 21. Jan between 17:30-18:30

Information

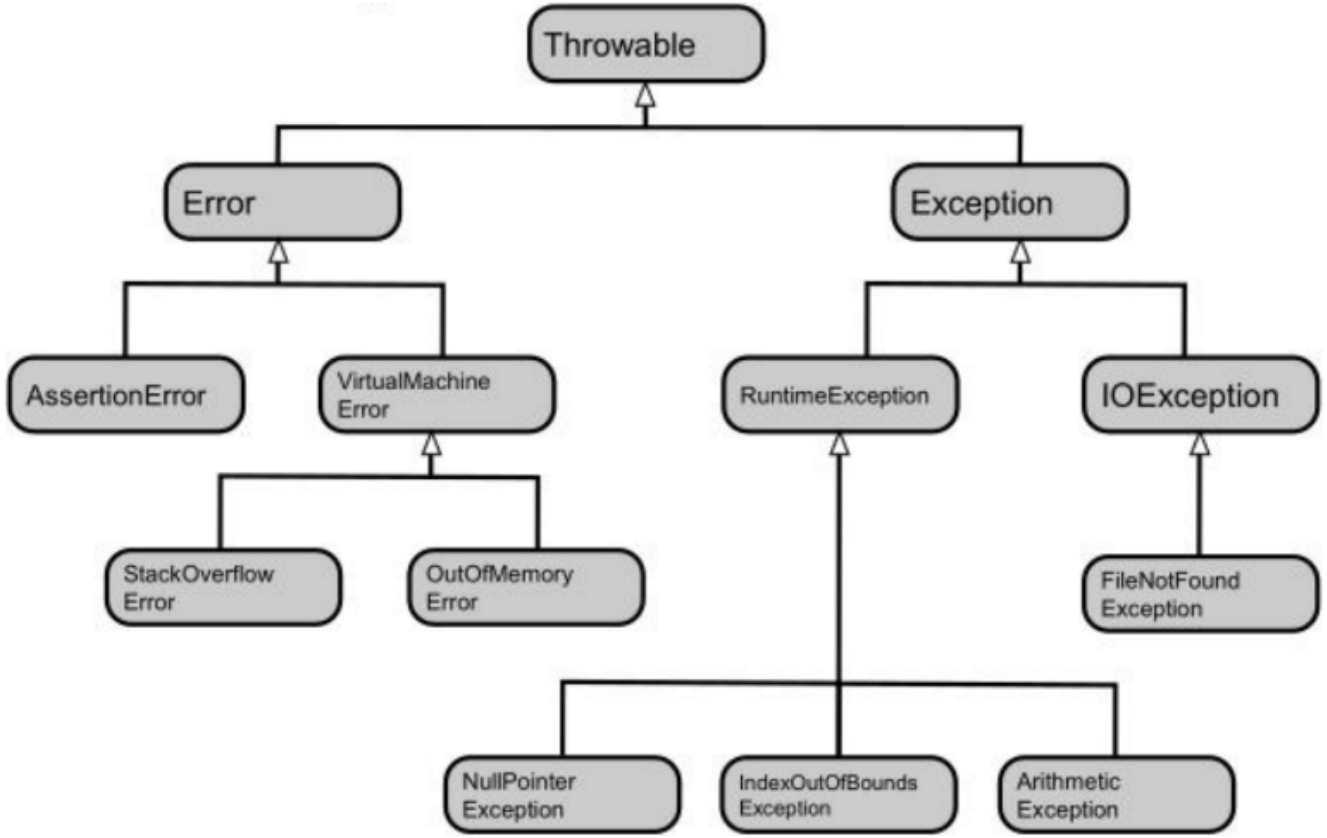
Präsenzübung

- **No other registration necessary**
  - The registration for Übungsschein is enough

- Don't forget to remember the number of tutorial you were initially assigned to
  - Number for this tutorial: **27**
- Try to get there at least 15 minutes beforehand
- Bring some ID with a picture (ID-Card, Residence permit..) **and** your student card
- Bring blue or black ink **pens**
- No aiding-material allowed (dictionaries allowed only if registered)

Last Week

Exceptions



Last Week

Errors vs Exceptions

Errors:

- Important problems
- Should never be handled in code

Exceptions:

- Handleable program mistakes
- Mostly discovered in execution

Last Week

Exceptions

- Never handle runtime exceptions → only `NumberFormatException` can be handled
- Don't catch the unspecific `Exception` class
- Never put try-catch block all around the program
- Avoid empty catch blocks
- Don't use exceptions as control flows

Last Week

Interfaces

```java
interface CanWalk {
    public void walk();
}

class Human implements CanWalk, CanTalk {
    @Override
    public void walk() {...}

    // ...
}

```

- **"can"** relations
- Will be **`implemented`** by classes

- Defines signatures to be implemented
- Single class can implement multiple interfaces

Last Week

Abstract vs. Interface

Abstract Class

- Only extend single abstract class
- Can have attributes and method bodies (base implementation)
- Contains partial implementation for common functionality

Interface

- Implement multiple interfaces
- Can't have attributes and method bodies
- Only creates an abstraction of the usage

Motivation

- Generics
- Recursions
- Revision

Generics

Problem

- You are modeling some base class *box* to use with different items
- You can put in a specific item into a declared box
  - i.e. for the pencil box you can put in only pencils

- How to define the **type** of items when modeling these boxes?
- Using `Object` would mean that each box can contain any object → not the goal

```
class Box {
    private List<???> items;

    public ??? getItem() { .. }

    public void addItem(??? item) { .. }
}

```

Generics

Idea

- Use generics by hinting the type (T)
- So the type also becomes a parameter for the class

```java
class Box<T> {
    private List<T> items;

    public T getItem() { .. }

    public void addItem(T item) { .. }
}
```

- This way can declare boxes of specific types:

```java

Box<Pencil> pencilBox = new Box<Pencil>();

pencilBox.add(pencil);
pencilBox.add(eraser);  // Doesn't work

Pencil item = pencilBox.getItem();

```

Generics

- The type parameter can be a class, an interface or an enum
  - JUST reference not object itself!

```
Pencil pencilObject;

class Box<pencilObject> {...} // NOT ALLOWED

class Box<Pencil> {...}

```

  - Can't be a primitive type

- Writing other letters doesn't change the type it makes it only more readible!
- Conventions follow:
  - T: For some class/type
  - E: Element
  - K: Key (e.g. Map)
  - V: Value (e.g. Map)

Generics

Example

Generic Type: `ArrayList<E>`

Type Parameter: `E`

Parameterised Type: `ArrayList<Student>`

Type Parameter: `Student`

Original Type: `ArrayList`

```java
class ArrayList<E> {
    public E get(int index) { .. }

    public E set(int index, E element) { .. }
}

void main() {
    List<Student> students = new ArrayList<Student>();

    // ...
    Student first = students.get(0);
}

```

Generics

Multiple Parameters

```java
public class Pair<T, S> {
    private T first;
    private S second;

    public Pair(T first, S second) {
        this.first = first;
        this.second = second;
    }

    public T getFirst() {
        return first;
    }

    public S getSecond() {
        return second;
    }
}

```

- Can use multiple type parameters

```java
void main() {
    String name = "my pencil";
    Pencil pencil = new Pencil();

    Pair<String, Pencil> pair = new Pair(name, pencil);
}

```

Generics

Bounded

- Can set a boundary to the type parameter
- Usage: <T extends Number>

```
class SpecialList<T extends Number> extends LinkedList<T> {
    private List<T> items;
    // ...

    public T get(int index) {
        // ...
    }
}

void main() {
    List<Double> myList = SpecialList<Double>();  // Allowed

    List<Student> students = SpecialList<Student>();  // Not allowed -> Student isn't a Number
}
```

Wildcards

```java
class Box<T> { ... }

int countItems ( Box<?> box ) {
    // Type of item doesn't matter
}

```

- Can also add boundaries to wildcards
- **`? extends Class`** : the type parameter must be a subclass of Class
- **`? super Class`** : the type parameter must be a superclass of Class

- Wildcards are used in method parameters
- Used when type parameter is not important

```java
int sumItems( Box<? extends Number> ) {
    // Type of item must be a number
}

int sumItems( Box<? super Integer> ) {
    // Type of item must be integer or super class
}

```

Recursion

Idea

- A method calls itself
- Has a **base case** and a **recursive case**

```
int factorial(int n) {
    // Base Case
    if (n == 0) { return 1; }

    // Recursive Case
    return n * factorial(n-1);
}

```

Recursion

Example

- Write the following function as a recursive method:

$$F(n) := \begin{cases} 0 & n = 0 \\ 1 & n = 1 \\ F(n-1) + F(n-2) & n \geq 2 \end{cases}$$

Recursion

Example

- Base case: `if` $(n \leq 1)$
- Recursive case: $F(n-1) + F(n-2)$

```
fib(n):

    if n <= 1:
        return n

    return fib(n-1) + fib(n-2)

```

Recursion

Example

- Result:

```
int fib(int n) {
    if (n <= 1) {
        return n;
    }

    return fib(n-1) + fib(n-2);
}

```

Recursion vs Iteration

Recursion:

- Number of calls unknown
- Many method calls $\rightarrow$ less efficient
- Hard to check if it terminates

Iteration:

- Number of calls/iterations known
- Easier to calculate and follow
- Easier to check if it terminates

OOP

- Add two attributes and two methods to following
- You don't have to implement the methods, leave it empty

```
class Car {
    // TODO: Implement me
}

```

Constructors

- Implement the following class
- Add missing methods for the attributes

```java
public class Student {
    private String id;
    private String name;

    public Student(String id, String name) {
        // TODO: Implement me
    }

    // Setters


    // Getters
}

```

Data Types

Which of the following are primitive?

- `Boolean`
- `List`
- `short`
- `int`
- `String`
- `Double`
- `Float`
- `long`
- `Enum`

Enums

- Write an enum that will represent the directions
- This enum should also store the value "degree" of that direction
- For example: direction North will have a degree 0

Control Structures

- Complete the following code
- Assume `values != null`
- Method should print out the **odd** values
- Use `System.out.println`

```
public static void printOdds(int[] values) {
    for (int i=___; ___; ___) {
        // TODO: Implement me
    }
}

```

Control Structures

- Fill the blank
- Method should return "true" if the statement is true and "false" if false

```
String getStatement(boolean a, boolean b) {
    return (a || b) ? ___ : ___
}

```

Control Structures

- Complete the following
- The loop should keep calling `expression()` until it returns `false`
- As the `expression()` is returning `true` the `currentValue` should be updated to the output of method `count()`

```
int currentValue = 0;
while ( ___ ) {
    // Implement
}
System.out.println(currentValue);

```

Control Structures

- Complete the following
- When `i` is even the method should increase `i` by 1
- When `i` is odd and bigger than 2 should print `"AAA"`

```
int doSomething(int i) {
    if ( ___ ) {



        System.out.println("AAA");
    }
    return i;
}

```

Dynamic Binding

- What is returned by the following calls?

```java
Human h0 = new Student();
h0.getName();
h0.getOccupation();

Student s0 = new CSStudent();
s0.getName();
s0.getOccupation();

CSStudent cs0 = new CSStudent();
cs0.getName();
cs0.getOccupation();

CSStudent cs1 = new Student();
cs1.getName();

```

```java
class Human {
    public String getName() { return "Tony"; }
}

class Student extends Human {
    public String getName() { return "Unlucky"; }
    public String getOccupation() { return "Student"; }
}

class CSStudent extends Student {
        public String getName() { return "Hustler"; }
}


```

Inheritance

- Which of the following calls will result in exception?

```java
Bird b0 = new Bird();
CanWalk w0 = new Bird();
CanFly f0 = new Bird();

b0.walk();
b0.talk();
b0.chirp();
b0.fly();
w0.walk();
w0.chirp();
w0.fly();
f0.talk();
f0.chirp();
f0.fly();

```

```java
interface CanWalk { void walk(); }
interface CanTalk { void talk(); }
interface CanFly { void fly(); }

class Bird implements CanWalk, CanFly {
    public void walk() {..}
    public void chirp() {..}
    public void fly() {..}
}

```

Recursion

- Write the following equation as a recursive method:

$$gcd(a, b) := \begin{cases} a & b = 0 \\ gcd(b, a \bmod b) & b \neq 0 \end{cases}$$

Conclusion

- Generics
- Recursions
- Revision

Reminder

Präsenzübung on 21. January between 17:30-18:30

Good luck in the exam!

OOP

```java
class Car {
    private String name;
    private String model;

    public void move() {
        // ...
    }

    public void turnOnLights() {
        // ...
    }
}

```

Constructors

```java
public class Student {
    private String id;
    private String name;

    public Student(String id, String name) {
        this.id = id;
        this.name = name;
    }

    // Setters
    public void setId(String id) {
        this.id = id;
    }

    public voic setName(String name) {
        this.name = name;
    }

    // Getters
    public String getId() {
        return this.id;
    }

    public String getName() {
        return this.name;
    }
}

```

Data Types

- short
- int
- long

Enums

```java
enum Direction {
    NORTH(0),
    EAST(90),
    WEST(270),
    SOUTH(180);

    private final int degree;

    private Direction(int degree) {
        this.degree = degree;
    }
}

```

Control Structures

```java
public static void printOdds(int[] values) {
    for (int i=0; i < values.length; i++) {
        int currentValue = values[i];
        if (currentValue % 2 == 1) {
            System.out.println(currentValue);
        }
    }
}

```

Control Structures

```
String getStatement(boolean a, boolean b) {
    return (a || b) ? "true" : "false"
}

```

Control Structures

```java
int currentValue = 0;
while (expression()) {
    currentValue = count();
}
System.out.println(currentValue);

```

Control Structures

```java
int doSomething(int i) {
    if (i % 2 == 0) {
        i++;
    } else if (i > 2) {
        System.out.println("AAA");
    }
    return i;
}

```

Dynamic Binding

```java
Human h0 = new Student();
h0.getName();                       // "Unlucky"
h0.getOccupation();                 // Error

Student s0 = new CSStudent();
s0.getName();                       // "Hustler"
s0.getOccupation();                 // "Student"

CSStudent cs0 = new CSStudent();
cs0.getName();                      // "Hustler"
cs0.getOccupation();                // "Student"

CSStudent cs1 = new Student();      // Error
cs1.getName();                      // Error

```

Inheritance

```java
Bird b0 = new Bird();
CanWalk w0 = new Bird();
CanFly f0 = new Bird();

b0.walk();
b0.talk();      // Error
b0.chirp();
b0.fly();
w0.walk();
w0.chirp();     // Error
w0.fly();       // Error
f0.talk();      // Error
f0.chirp();     // Error
f0.fly();

```

Recursion

```
int gcd(int a, int b) {
    if (b == 0) {
        return a;
    }

    return gcd(b, a % b);
}

```